

实验报告

分布式影响力最大化中的子模优化与贪心近似算法

陈亦新

张弼弛

叶瑶勤

CS240: 算法设计与分析, 2026 年春季学期

摘要

本项目研究影响力最大化问题：在网络中选择少量初始种子节点，使信息、行为或干预经过扩散后覆盖尽可能多的节点。按照课程项目要求，我们将现代网络数据应用与经典算法联系起来，重点讨论单调子模最大化、贪心近似、懒惰求值，以及带通信约束的分布式候选节点共享方法。项目实现了独立级联模型模拟、随机/度数/PageRank 基线、中心化蒙特卡洛贪心、CELF 懒惰贪心，以及 GreeDI 风格的分布式局部候选合并算法。实验包含快速小规模基准和更接近最终状态的大规模预设。当前结果表明，在加权级联概率规则下，结构性基线非常强；CELF 在大规模预设上保持接近贪心的质量，同时将平均影响力估计次数从 1026.5 降到 266.2；分布式候选共享则展示了质量、运行时间和通信预算之间的权衡。

1 背景与相关工作

影响力最大化是社交网络、病毒式营销、公共卫生干预和虚假信息分析中的经典问题。给定一个网络和有限预算，我们希望选择若干种子节点，使扩散过程最终激活的节点数量尽可能多。这个问题不是单纯的数据分析问题，而是带随机扩散过程的组合优化问题。

Kempe、Kleinberg 和 Tardos [1] 在独立级联模型 (IC) 和线性阈值模型 (LT) 下系统提出了影响力最大化问题，证明其为 NP-困难，并证明期望影响范围函数具有单调性和子模性。Nemhauser、Wolsey 和 Fisher 的经典结论 [2] 说明，在基数约束下最大化非负单调子模函数时，贪心算法具有 $(1 - 1/e)$ 近似保证。

实际困难在于，精确计算影响范围通常代价很高。朴素实现需要对大量候选种子集合做蒙特卡洛扩散模拟。Leskovec 等人 [3] 提出的 CELF 方法利用子模函数边际收益递减性质，跳过大量不必要的重新估计。对于分布式场景，Mirzasoleiman 等人 [4] 提出的 GreeDI 方法将元素集合划分到多个机器上，先局部贪心，再合并局部代表元素。近年工作说明可扩展性仍然是活跃算法问题：TIM/IMM [5, 6] 使用反向影响采样，PaC-IM [7] 压缩并并行化影响力最大化草图，Chen 等人 [8] 研究 MapReduce 和自适应复杂度模型下的分布式子模最大化，GreedIRIS [9] 将反向影响采样与分布式流式最大覆盖结合。本项目将这些系统作为通信受限问题的动机，但实现重点放在透明、可复

现的 Greedy 风格方案上。

2 问题形式化

设 $G = (V, E, p)$ 为有向图，每条边 (u, v) 具有传播概率 $p_{uv} \in [0, 1]$ 。给定种子预算 k ，影响力最大化问题为

$$\max_{S \subseteq V, |S| \leq k} \sigma(S),$$

其中 $\sigma(S)$ 表示以 S 为初始种子集合时，扩散结束后的期望激活节点数。

本项目主要使用独立级联模型。节点 u 第一次被激活后，对每个尚未激活的出邻居 v 有且仅有一次激活机会，成功概率为 p_{uv} 。所有边上的激活尝试相互独立，直到没有新的激活节点为止。

在 IC 模型下， σ 是单调子模函数：

$$\sigma(A \cup \{v\}) - \sigma(A) \geq \sigma(B \cup \{v\}) - \sigma(B), \quad A \subseteq B, v \notin B.$$

这种边际收益递减性质是贪心算法具有理论保证的原因，也是 CELF 能够把旧边际收益作为当前上界的原因。

3 代表性方法分析

3.1 中心化贪心

中心化贪心算法从 $S_0 = \emptyset$ 开始。在第 t 轮选择

$$v^* = \arg \max_{v \in V \setminus S_t} (\hat{\sigma}(S_t \cup \{v\}) - \hat{\sigma}(S_t)),$$

其中 $\hat{\sigma}$ 为蒙特卡洛模拟得到的 IC 影响范围估计。随后令 $S_{t+1} = S_t \cup \{v^*\}$ ，直到 $|S| = k$ 。

若图有 n 个节点、 m 条边，一次 IC 模拟最坏情况下需要 $O(n + m)$ 时间。若每次影响力估计使用 R 次模拟，则朴素贪心的代价约为

$$O(knR(n + m)).$$

这在课程项目的小图上可以运行，但在大规模网络上很快变得昂贵。

3.2 CELF 懒惰贪心

CELF 用优先队列维护候选节点的缓存边际收益。由于子模性保证种子集合增大后某个节点的边际收益不会增加，旧边际收益可以作为当前边际收益的上界。当队首节点的边际收益已经是在当前种子集合大小下计算得到时，算法可以直接选择它；否则只重新计算这一个节点并放回队列。CELF 目标上仍然对应贪心选择，但通常显著减少影响力估计次数。

3.3 GreeDI 风格分布式贪心

分布式部分将节点集合划分为 m 个局部部分。第 i 个工作节点在本地候选节点上运行贪心，并返回最多 q 个候选节点。协调器形成候选池

$$C = \bigcup_{i=1}^m C_i,$$

再只在 C 上运行最终贪心选择。本项目测试 $m \in \{2, 4, 8\}$ 和 $q \in \{k, 2k, 5k\}$ 。通信代价用传输候选节点数量近似，即约为 mq ，但受每个分区大小上限限制。

4 复现与实现

代码结构如下：

- `src/simulation.py`: 独立级联模拟与蒙特卡洛影响范围估计；
- `src/baselines.py`: 随机、加权出度和 PageRank 种子选择；
- `src/greedy.py`: 中心化贪心与 CELF 懒惰贪心；
- `src/distributed.py`: 随机节点划分与 GreeDI 风格候选共享；
- `src/graphs.py`: 基准图构造与加权级联概率设置；
- `src/experiments.py`: 端到端实验运行与 CSV 结果输出。

由于完整复现大规模影响力最大化系统超出了课程项目范围，我们复现的是核心算法机制：IC 扩散估计、贪心子模选择、CELF 懒惰求值，以及分布式局部候选合并。最终代码支持两套基准预设。快速小规模预设使用 $n \in \{40, 70\}$ 的 Erdos–Renyi、Barabasi–Albert 和 Watts–Strogatz 图，并包含 34 节点的 Zachary Karate Club 图。大规模预设使用同三类合成图的 $n \in \{50, 100, 200\}$ 规模，并同样包含 Karate 图。大规模合成图使用 $k = 10$ ，Karate 使用 $k = 5$ 。当前提交的 CSV 结果保存在输出目录中，文件名为 `influence_results.csv` 和 `influence_results_large.csv`。

图 1 展示了快速小规模预设中使用的基准图结构。加入这张图是因为图拓扑会直接影响不同种子选择启发式的表现：Erdos–Renyi 图强调随机连边，Barabasi–Albert 图包含枢纽节点，Watts–Strogatz 图强调局部小世界结构，Karate 图则提供一个小型真实社交网络。

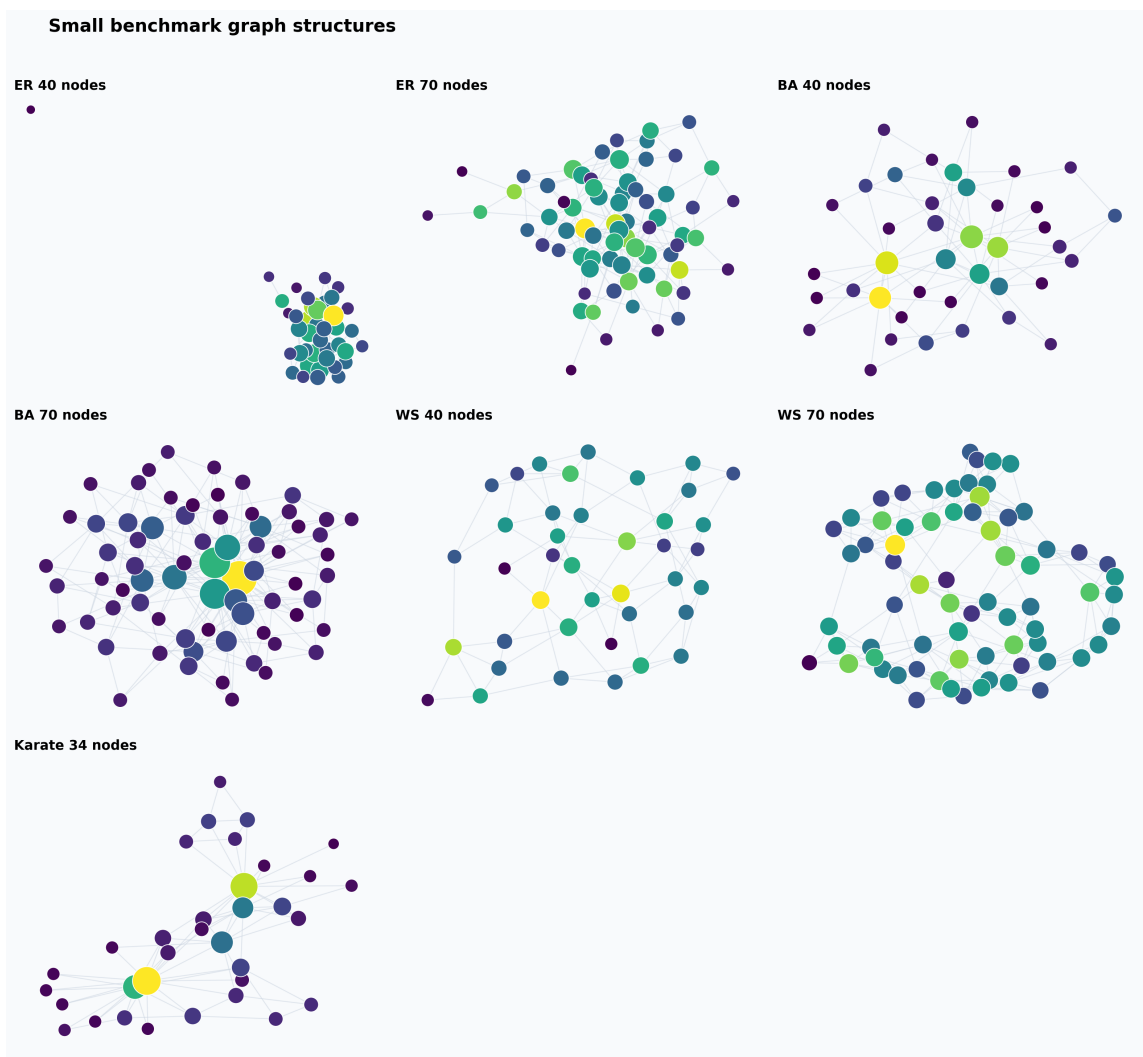


图 1: 快速小规模基准图结构，节点颜色表示加权出度。

为了提高不同算法之间的可比性，每个算法得到最终种子集合后，都使用同一个评估随机种子重新估计影响范围。因此报告中的 `estimated_spread` 不是算法选择过程中内部使用的估计值，而是最终种子集合的公平再评估结果。

5 实验评估

正文嵌入了图结构总览，以及每个主要指标的一张图：影响范围、相对质量、传输候选数量、影响力估计次数和运行时间。实验脚本还会在 `figures/comparisons/` 下生成横向和纵向柱状比较图；这些图没有全部放入正文，因为它们只是用另一种布局展示同一组指标，全部加入会造成重复。

5.1 整体算法比较

表 1 汇总了当前大规模预设十个图实例上的平均结果。其中质量比表示在同一图实例上、经过公平再评估后，相对于中心化贪心结果的比例。小规模预设仍作为快速检查保留，但大规模预设更能代表最终项目状态。

表 1: 所有基准实例上的平均性能。

算法	影响范围	质量比	运行时间 (s)	估计次数
随机	31.59	0.79	0.00	1.0
加权出度	43.71	1.07	0.01	1.0
PageRank	35.19	0.89	0.00	1.0
贪心	40.43	1.00	4.67	1026.5
CELF	40.05	0.99	0.77	266.2

一个需要诚实说明的现象是：大规模预设中，贪心并没有稳定超过结构性基线。加权出度表现尤其强，因为加权级联概率规则本身会让结构中心位置节点具有较高影响潜力。这并不构成对贪心近似定理的反例：理论比较的是精确子模目标下的贪心解与最优解，而本项目实现中选择阶段使用的是带噪声的蒙特卡洛估计，最终又用另一个随机种子进行公平再评估。

CELF 的表现符合预期：平均影响范围与贪心接近，质量比为 0.99，同时将影响力估计次数从 1026.5 降到 266.2，并将平均运行时间从 4.67 秒降到 0.77 秒。这是本次复现中最清晰的理论到实践结果。

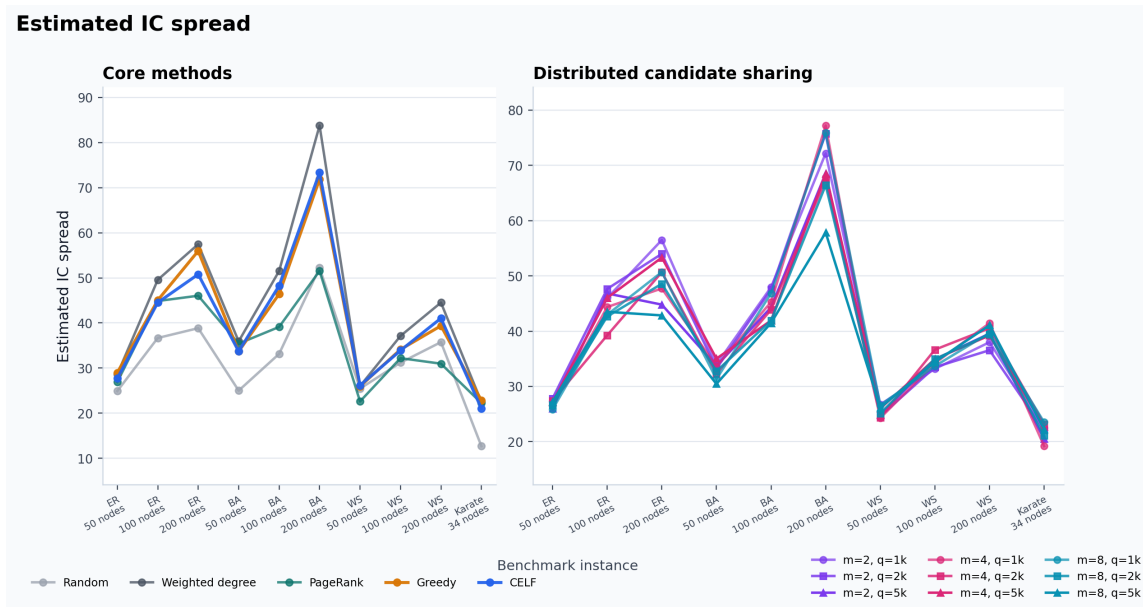


图 2: 大规模预设下的 IC 影响范围估计。

5.2 分布式通信权衡

表 2 汇总了大规模预设下的分布式变体。记号 $(m, q/k)$ 表示 m 个分区以及本地候选预算 $q = (q/k) \cdot k$ 。

表 2: 不同分区数和通信预算下的平均分布式性能。

设置	影响范围	质量比	运行时间 (s)	估计次数	候选数
(2, 1)	40.43	1.00	1.54	1130.5	19.0
(2, 2)	40.28	0.99	3.78	2114.5	38.0
(2, 5)	38.40	0.95	9.89	3984.1	78.4
(4, 1)	39.39	0.96	1.54	1234.5	38.0
(4, 2)	38.56	0.96	3.43	2072.1	66.4
(4, 5)	39.48	0.98	6.44	3068.1	108.4
(8, 1)	39.70	0.98	1.66	1362.1	66.4
(8, 2)	37.95	0.94	2.86	1924.9	96.4
(8, 5)	36.72	0.93	3.12	2080.9	108.4

分布式结果清楚展示了成本侧的权衡：更大的本地预算会增加传输候选节点数量，并通常带来更多影响力估计。质量侧并不单调。大规模预设中平均表现最好的分布式设置是 $(m, q/k) = (2, 1)$ ，经过公平再评估后几乎与中心化贪心持平。增大 q 并不会在这次实验中自动改善质量，因为本地选择和协调器选择仍然依赖带噪声的蒙特卡洛估计。更合理的解释是：较小候选池已经可以有竞争力，而额外通信有真实计算成本，并且需要更稳定的估计才能持续发挥作用。

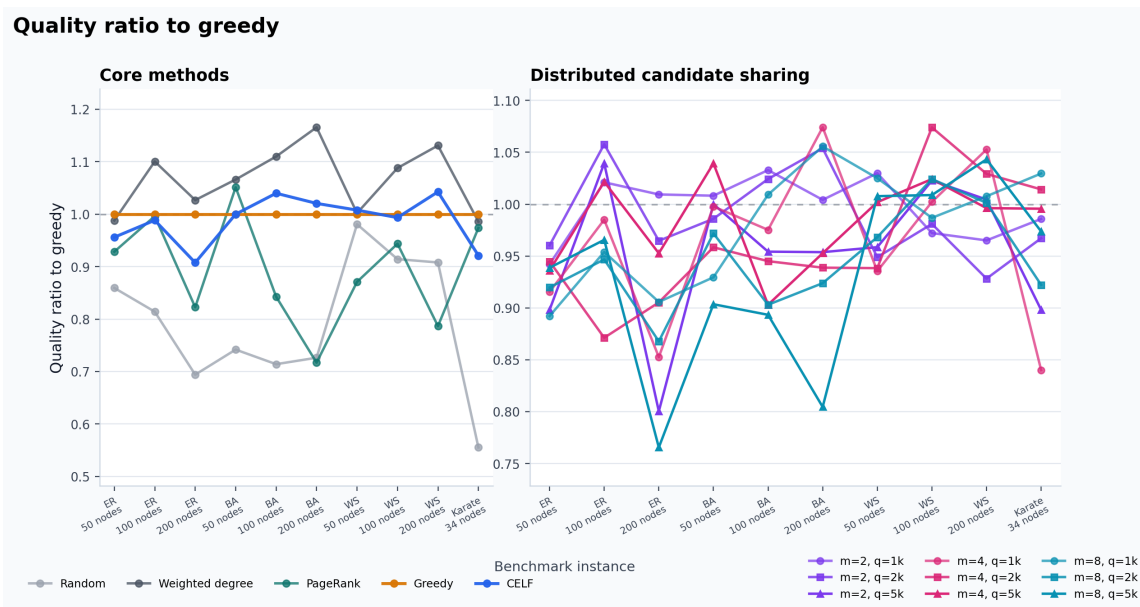


图 3: 大规模预设下相对于中心化贪心的质量比例。

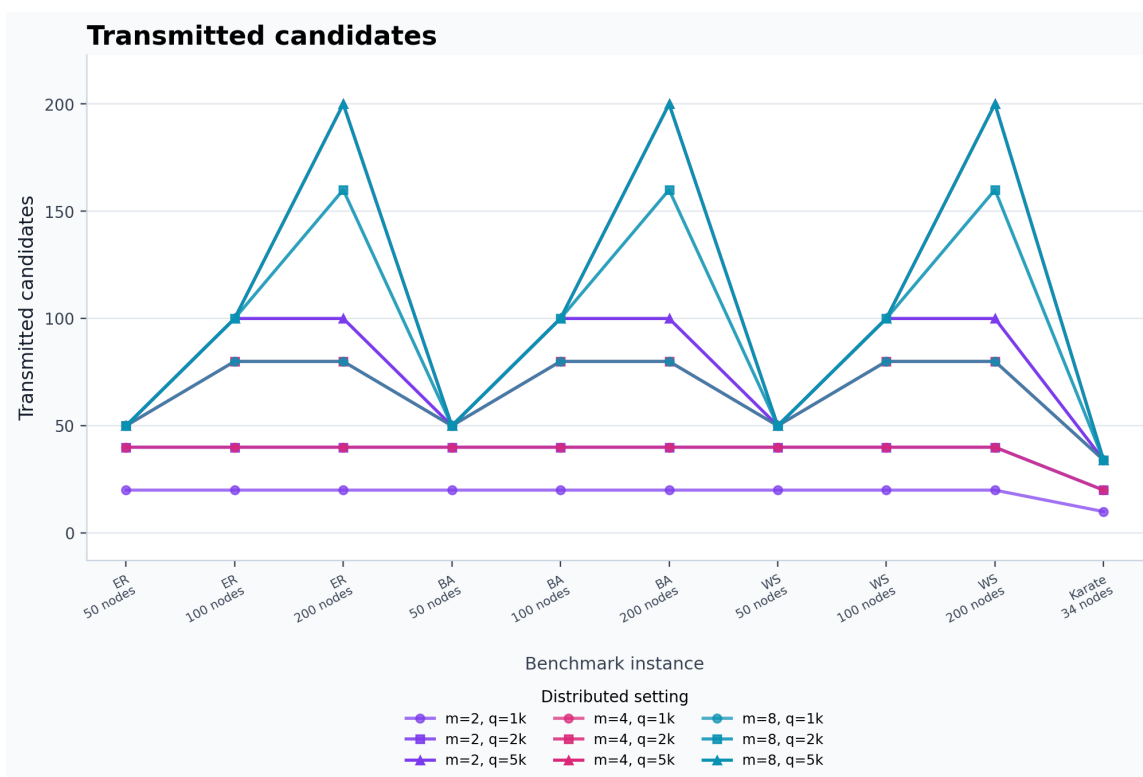


图 4: 大规模预设下的通信代价近似: 传输候选节点数量。

5.3 运行时间与估计次数

在大规模预设上，中心化贪心的成本已经更明显：平均影响力估计次数达到 1026.5。CELF 通过复用边际收益上界，将该数值降到 266.2。分布式变体总估计次数更高，因为每个工作节点运行一轮本地贪心，协调器还要运行一轮最终贪心。只有当工作节点可以并行执行，或者完整图候选搜索因为内存、数据归属、隐私或通信约束而不可行时，这种额外计算才是合理的。当前实现模拟了通信受限的算法结构，但并没有真正并行执行工作节点。

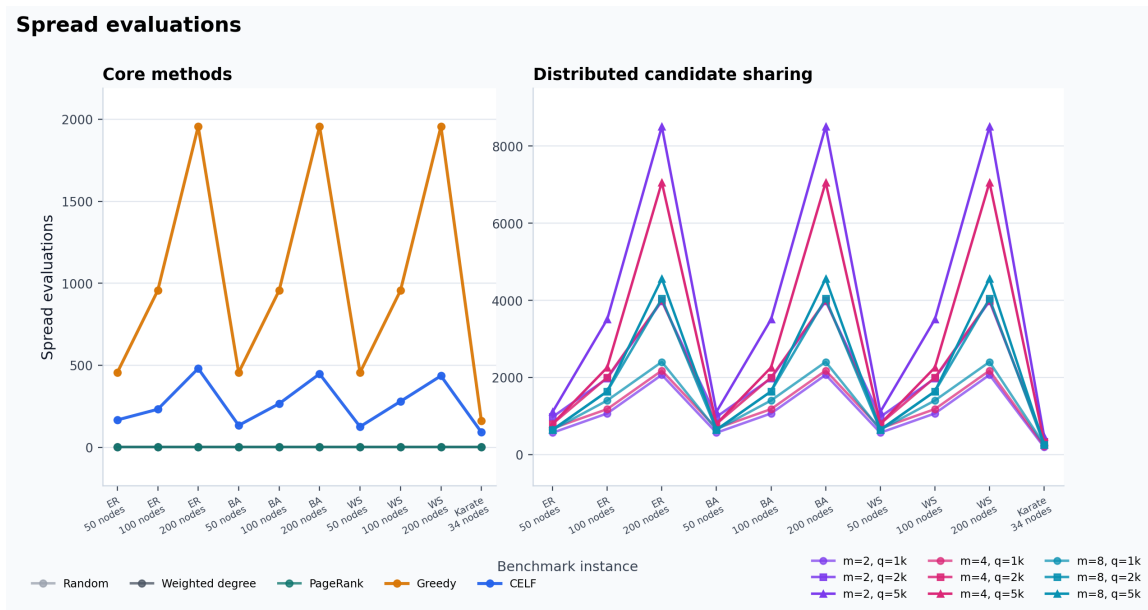


图 5: 大规模预设下的影响力估计次数。CELF 相比朴素贪心减少了估计次数。

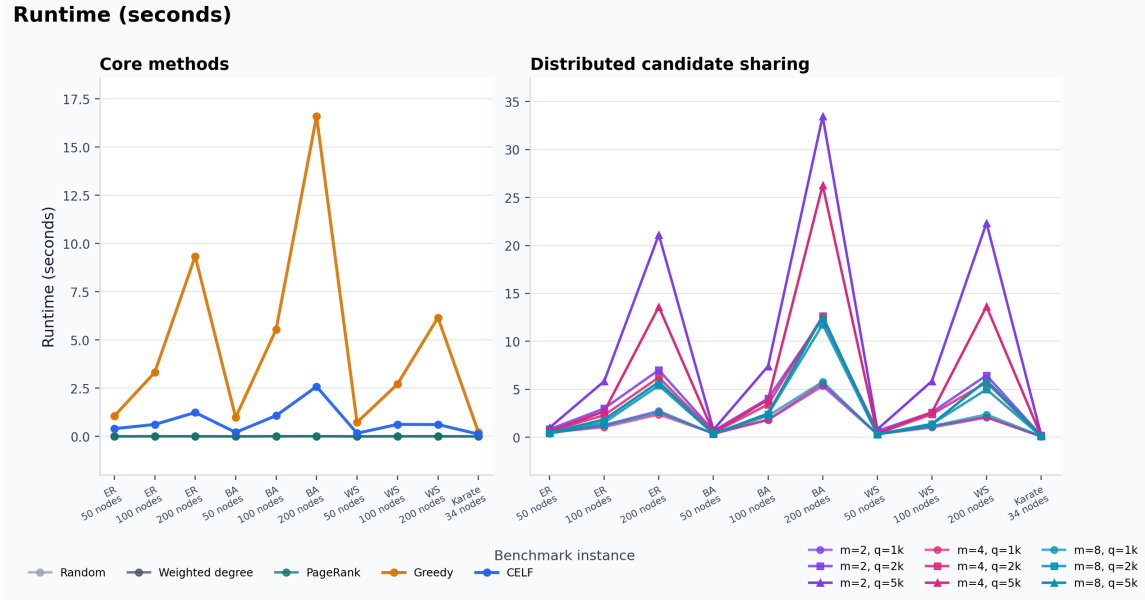


图 6: 大规模预设下的实际运行时间。

6 偏差、局限与可选拓展

项目计划中提到 $n \in \{500, 1000, 5000\}$ 的更大合成图和 Wiki-Vote、NetHEPT 等真实网络。最终复现使用最大 200 节点的课程规模合成图以及 Karate 图，是为了让蒙特卡洛贪心实验能在课程项目环境中稳定运行。这使实现更透明，但限制了强可扩展性结论。实验入口现在支持通过 `--sizes` 自定义合成图规模，但提交的 CSV 输出采用文档中说明的小规模和大规模预设。

另一个限制是蒙特卡洛噪声。为了控制运行时间，当前实验使用的模拟次数较少，分布式局部选择阶段使用的模拟次数还进一步降低。因此最终种子集合可能对随机性较敏感。这解释了为什么公平再评估有时会让加权出度、CELF 或分布式变体高于中心化贪心。更强的最终实验应增加模拟次数，在多个随机种子上重复实验，并报告置信区间。

本项目已经实现的可选拓展是 GreeDI 风格的通信受限变体，同时加入了小规模/大规模两套预设和图结构可视化。进一步可做的拓展包括确定性图划分策略、更大真实数据集、基于 RR 集合的最大覆盖方法，以及真正并行执行工作节点以测量分布式加速效果。

7 结论

本项目展示了如何用经典算法理解现代网络问题。IC 扩散模型下的影响力最大化具有单调子模目标；贪心选择提供近似保证；CELF 将子模性转化为实际加速；GreeDI 风格候选共享则把问题连接到通信受限的分布式优化。虽然实验仍属于课程规模，但已经复现了核心算法机制，并揭示了一个重要实践问题：影响力最大化的效果不仅取决于近似算法本身，也取决于扩散估计精度、图结构

和通信限制。

代码可用性

本项目的完整源代码、实验脚本、图表以及报告源文件已在以下 GitHub 仓库公开: https://github.com/Snake-Konginchrist/CS240_Course_Project。

参考文献

- [1] D. Kempe, J. Kleinberg, and E. Tardos. Maximizing the spread of influence through a social network. *KDD*, 2003.
- [2] G. L. Nemhauser, L. A. Wolsey, and M. Fisher. An analysis of approximations for maximizing submodular set functions. *Mathematical Programming*, 1978.
- [3] J. Leskovec, A. Krause, C. Guestrin, C. Faloutsos, J. VanBriesen, and N. Glance. Cost-effective outbreak detection in networks. *KDD*, 2007.
- [4] B. Mirzasoleiman, A. Karbasi, R. Sarkar, and A. Krause. Distributed submodular maximization: Identifying representative elements in massive data. *NeurIPS*, 2013.
- [5] Y. Tang, X. Xiao, and Y. Shi. Influence maximization: Near-optimal time complexity meets practical efficiency. *SIGMOD*, 2014.
- [6] Y. Tang, Y. Shi, and X. Xiao. Influence maximization in near-linear time: A martingale approach. *SIGMOD*, 2015.
- [7] L. Wang, X. Ding, Y. Gu, and Y. Sun. Fast and space-efficient parallel algorithms for influence maximization. *PVLDB*, 2023.
- [8] Y. Chen, T. Dey, and A. Kuhnle. Scalable distributed algorithms for size-constrained submodular maximization in the MapReduce and adaptive complexity models. *JAIR*, 2024.
- [9] R. Barik, W. Cappa, S. M. Ferdous, M. Minutoli, M. Halappanavar, and A. Kalyanaraman. GreediRIS: Scalable influence maximization using distributed streaming maximum cover. *Journal of Parallel and Distributed Computing*, 2025.